



World Model

中期汇报

汇报人: Codex

日期: 2026.05.24



目录

01

研究背景

为什么要学习环境的内部模型

02

方法框架

V-M-C 三模块与训练流程

03

实验结果

CarRacing 与 Doom 场景验证

04

问题与计划

误差累积、代码分析和后续安排

01

研究背景



World Model 的直觉

人类并不是只对当前观测做反应，而是在脑中构建一个可预测的世界。World Model 的目标也是类似的：把高维观测压缩为潜变量，并在潜空间中预测未来状态，从而让智能体在“想象”中规划行为。

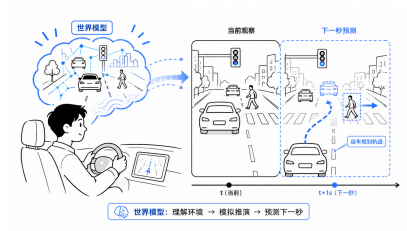


类比：先形成世界表征，再基于表征做决策。

任务场景：从像素到行动



Car Racing 这类任务输入是连续图像，决策需要理解道路、车辆位置和未来轨迹。



World Model 把“观测、记忆、控制”拆开，让模型先学习环境动力学，再训练控制策略。

传统 RL 与 World Model RL



核心差异：传统 RL 更依赖真实环境交互；World Model RL 先学习可预测的环境模型，再在模型内部生成轨迹和训练策略。

02

方法框架



V-M-C 总体架构

At each time step, our agent receives an **observation** from the environment.

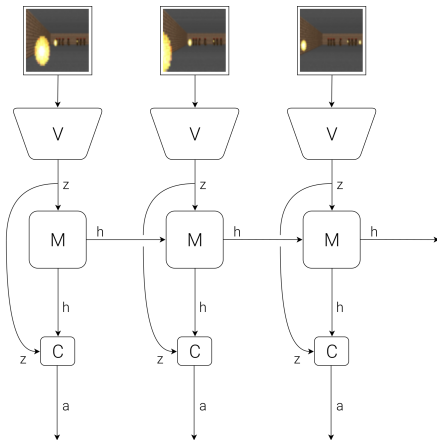
World Model

The **Vision Model (V)** encodes the high-dimensional observation into a low-dimensional latent vector.

The **Memory RNN (M)** integrates the historical codes to create a representation that can predict future states.

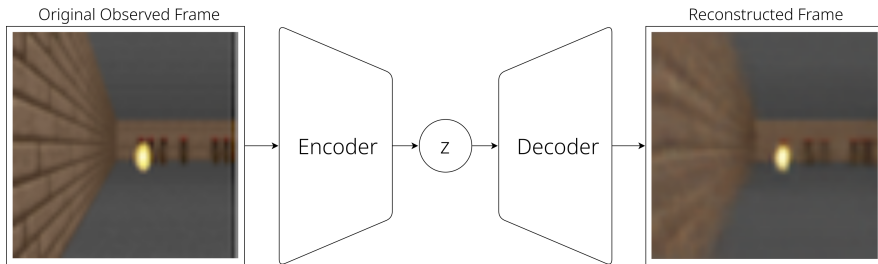
A small **Controller (C)** uses the representations from both **V** and **M** to select good actions.

The agent performs **actions** that go back and affect the environment.



Vision Model 负责压缩观测, Memory Model 负责预测潜状态演化, Controller 根据潜变量和记忆状态输出动作。

Vision Model: 学习低维表征



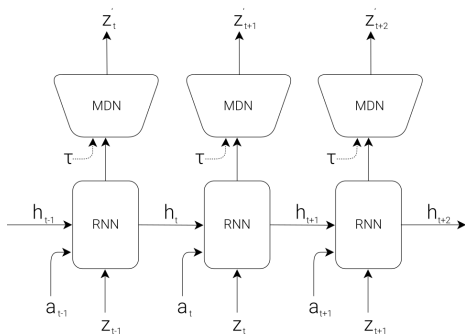
作用

把原始图像编码成低维潜变量，降低后续模型学习难度。

验收标准

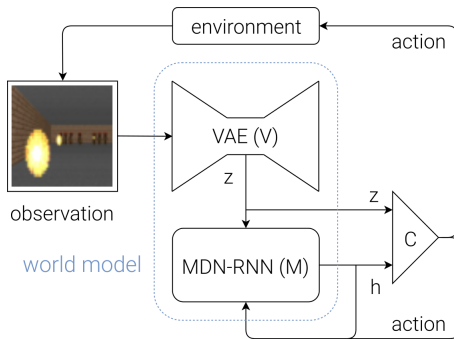
重建图像应保留道路、障碍和视野结构，避免压缩后丢失关键控制信息。

Memory 与 Controller



Memory Model

用 RNN 和混合密度网络预测未来潜变量分布，刻画环境不确定性。

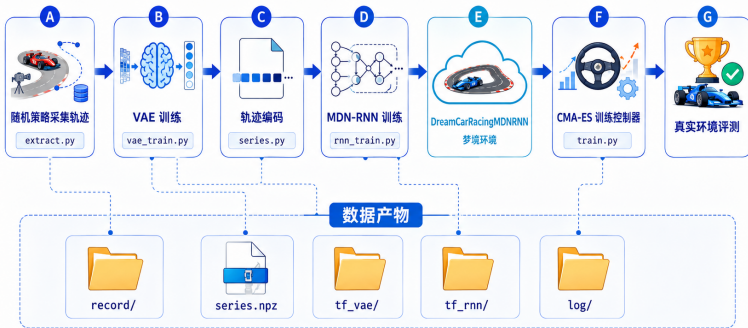


Controller

基于当前潜变量和记忆状态输出动作，通常可以用较小规模策略网络实现。

训练流程

World Models 训练流程图



流程包括数据采集、VAE 训练、MDN-RNN 训练、Controller 优化和模型评估。

03

实验结果



实验设置与可视化



CarRacing

在连续控制环境中观察模型能否捕捉道路结构和动作反馈。



Doom

在更复杂视觉场景中评估模型对局部状态和奖励信号的建模能力。

训练曲线与生成效果



当前结果显示模型能学习部分动态规律，但稳定性和泛化能力仍需要系统化验证。

04

问题与计划



误差滚雪球与代码定位



```
53 def reset(self):
54     self.rnn_states = rnn_init_state(self.rnn)
55     result = super(CarRacingMDPNN, self).reset()
56     self._utils = len(self.track)
57     return result
58 def step(self, action):
59     obs, reward, done, info = super(CarRacingMDPNN, self).step(action)
60     z = tf.squeeze(self.encode_obs(obs))
61     h = tf.squeeze(self.rnn_states[0])
62     c = tf.squeeze(self.rnn_states[1])
63     if self.rnn.args.state_space == 2:
64         z_state = tf.concat([z, c, h], axis=-1)
65     else:
66         z_state = tf.concat([z, h], axis=-1)
67     if action is not None: # don't compute state on reset
68         self.rnn_states = rnn_next_state(self.rnn, z, action, self.rnn_states)
69     if self.with_obs:
70         return [z_state, obs], reward, done, {}
71     else:
72         return z_state, reward, done, {}
```

```
54 def ds_gen():
55     for _ in range(args.rnn_num_steps):
56         indices = np.random.permutation(N_data)[0:args.rnn_batch_size]
57         # suboptimal b/c we are always only taking first set of steps
58         mu = data_mu[indices][:, :args.rnn_max_seq_len]
59         logvar = data_logvar[indices][:, :args.rnn_max_seq_len]
60         action = data_action[indices][:, :args.rnn_max_seq_len]
61         z = sample_vae(mu, logvar)
62         r = tf.cast(data_r[indices], tf.float16)[: , :args.rnn_max_seq_len]
63         d = tf.cast(data_d[indices], tf.float16)[: , :args.rnn_max_seq_len]
64         N = tf.cast(data_N[indices], tf.float16)[: , :args.rnn_max_seq_len]
65         yield z, action, r, d, N
```

主要风险是一步预测的小误差在多步 rollout 中持续放大。下一步需要结合可视化、代码路径和定量指标定位误差来源。

后续研究计划



优先推进三件事：稳定训练流程、补充消融实验、建立从观测重建到控制表现的评估闭环。



谢谢聆听

World Model Midterm Report

汇报人：Codex

日期：2026.05.24

